



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА - Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра математического обеспечения и стандартизации
информационных технологий (МОСИТ)

ИТОГОВЫЙ РАБОТЕ

по дисциплине «Тестирование и верификация программного
обеспечения»

Модульное и мутационное тестирование программного продукта

Студенты группы *ИКБО-43-23, Буркеев Р.Р., Данюков
К.А., Жаворонкова А.А., Савельев М.С.*

(подпись)

Преподаватель *Ильичев Г.П.*

(подпись)

Отчет
представлен *«10» октября 2025г.*

Москва, 2025

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
Цель.....	3
Задачи.....	3
ГЛАВА 1. РАЗРАБОТКА МОДУЛЯ	4
1.1. Описание модуля.....	4
1.2. Описание функциональности	4
1.3. Исходный код созданного модуля (Генератор паролей).....	4
1.4 Исходный код тестируемого модуля (Конвертер единиц измерения)	6
ГЛАВА 2. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ	9
2.1 Введение	9
2.2 Подход к тестированию	9
2.3 Исходный код модульного тестирования	10
ГЛАВА 3. МУТАЦИОННОЕ ТЕСТИРОВАНИЕ	18
3.1 Введение	18
3.2 Подход к тестированию	18
3.3 Исходный код мутационного тестирования	19
3.4 Типы мутационных операторов.....	25
3.5 Результаты тестирования	26
3.6 Анализ выживших мутантов.....	28
ГЛАВА 4. ВЫВОД.....	29
4.1 Модульное тестирование	29
4.2 Мутационное тестирование	29
4.3 Анализ недостатков и их устранение	29
4.4 Итоговые выводы	30
ЗАКЛЮЧЕНИЕ	31

ВВЕДЕНИЕ

Цель

Познакомиться с процессом модульного и мутационного тестирования, включая разработку, проведение тестов, исправление ошибок, анализ тестового покрытия, а также оценку эффективности тестов путём применения методов мутационного тестирования.

Задачи

- изучить основы модульного тестирования и его основные принципы;
- освоить использование инструментов для модульного тестирования (pytest для Python, JUnit для Java и др.);
- разработать модульные тесты для программного продукта и проанализировать их покрытие кода;
- изучить основы мутационного тестирования и освоить инструменты для его выполнения (MutPy, PIT, Stryker);
- применить мутационное тестирование к программному продукту, оценить эффективность тестов;
- улучшить существующий набор тестов, ориентируясь на результаты мутационного тестирования;
- оформить итоговый отчёт с результатами проделанной работы.

ГЛАВА 1. РАЗРАБОТКА МОДУЛЯ

1.1. Описание модуля

Модуль - Генератор и валидатор паролей. Данный модуль предоставляет функциональность для генерации случайных паролей заданной длины, а также для проверки паролей на соответствие требованиям безопасности. Модуль включает в себя функции для проверки длины пароля, наличия заглавных букв и цифр.

1.2. Описание функциональности

Функциональным назначением программы является автоматизация процесса создания безопасных паролей и их последующей проверки на соответствие стандартным требованиям безопасности.

1.3. Исходный код созданного модуля (Генератор паролей)

Предоставлен исходный код данного модуля

```

import random
import string

def generate_password(Length=8):
    """Генерирует случайный пароль"""
    chars = string.ascii_letters + string.digits
    password = ''
    for i in range(Length):
        password += random.choice(chars)
    return password

def check_length(password, min_length=6):
    """Проверяет длину пароля"""
    return len(password) >= min_length

def has_uppercase(password):
    """Проверяет наличие заглавных букв"""
    for char in password:
        if char.isupper():
            return True
    return False

def has_digits(password):
    """Проверяет наличие цифр"""
    for char in password:
        if char.isdigit():
            return True
    return False

def validate_password(password):
    """
    Функция с преднамеренной ошибкой
    Проверяет пароль на соответствие правилам
    """
    # Правило 1: длина не менее 6 символов
    if len(password) < 6:
        return False

    # Правило 2: должна быть хотя бы одна заглавная буква
    if not has_uppercase(password):
        return False

    # Правило 3: должна быть хотя бы 1 цифра
    has_digit = False
    for char in password:
        if char == '1':
            has_digit = True
            break

    if not has_digit:
        return False

    return True

```

Рисунок 1 - Исходный код генератора паролей

1.4 Исходный код тестируемого модуля (Конвертер единиц измерения)

```
def celsius_to_fahrenheit(celsius):
    """
    Конвертирует температуру из Цельсия в Фаренгейт

    Args:
        | celsius (float): Температура в градусах Цельсия

    Returns:
        | float: Температура в градусах Фаренгейта
    """
    return (celsius * 9 / 5) + 32

def fahrenheit_to_celsius(fahrenheit):
    """
    Конвертирует температуру из Фаренгейта в Цельсий

    Args:
        | fahrenheit (float): Температура в градусах Фаренгейта

    Returns:
        | float: Температура в градусах Цельсия
    """
    return (fahrenheit - 32) * 5 / 9

def meters_to_feet(meters):
    """
    Конвертирует длину из метров в футы

    Args:
        | meters (float): Длина в метрах

    Returns:
        | float: Длина в футах
    """
    return meters * 3.28084

def feet_to_meters(feet):
    """
    Конвертирует длину из футов в метры

    Args:
        | feet (float): Длина в футах

    Returns:
        | float: Длина в метрах
    """
    return feet / 3.28084

def kilograms_to_pounds(kilograms):
    """
    Конвертирует массу из килограммов в фунты

    Args:
        | kilograms (float): Масса в килограммах

    Returns:
        | float: Масса в фунтах
    """
    return kilograms * 2.20462 # Должно быть 2.20462

def pounds_to_kilograms(pounds):
    """
    Конвертирует массу из фунтов в килограммы

    Args:
        | pounds (float): Масса в фунтах

    Returns:
        | float: Масса в килограммах
    """
    return pounds / 2.20462
```

Рисунок 2 - Исходный код конвертера

```

def kilometers_to_miles(kilometers):
    """
    Конвертирует расстояние из километров в мили

    Args:
        kilometers (float): Расстояние в километрах

    Returns:
        float: Расстояние в милях
    """
    return kilometers * 0.621371

def miles_to_kilometers(miles):
    """
    Конвертирует расстояние из миль в километры

    Args:
        miles (float): Расстояние в милях

    Returns:
        float: Расстояние в километрах
    """
    return miles / 0.621371

def liters_to_gallons(liters):
    """
    Конвертирует объем из литров в галлоны

    Args:
        liters (float): Объем в литрах

    Returns:
        float: Объем в галлонах
    """
    return liters * 0.264172

def gallons_to_liters(gallons):
    """
    Конвертирует объем из галлонов в литры

    Args:
        gallons (float): Объем в галлонах

    Returns:
        float: Объем в литрах
    """
    return gallons / 0.264172

def display_conversion_menu():
    """Отображает меню доступных конвертаций"""
    print("\n=== Конвертер единиц измерения ===")
    print("1. Температура: Цельсий -> Фаренгейт")
    print("2. Температура: Фаренгейт -> Цельсий")
    print("3. Длина: Метры -> Футы")
    print("4. Длина: Футы -> Метры")
    print("5. Масса: Килограммы -> Фунты")
    print("6. Масса: Фунты -> Килограммы")
    print("7. Расстояние: Километры -> Мили")
    print("8. Расстояние: Мили -> Километры")
    print("9. Объем: Литры -> Галлоны")
    print("10. Объем: Галлоны -> Литры")
    print("0. Выход")

def get_user_input():
    """
    Получает выбор пользователя и значение для конвертации

    Returns:
        tuple: (выбор_меню, значение_для_конвертации)
    """
    try:
        choice = int(input("Выберите тип конвертации (0-10): "))
        if choice == 0:
            return 0, 0

        value = float(input("Введите значение для конвертации: "))

```

Рисунок 3 - Продолжение кода конвертера

```

        return choice, value
    except ValueError:
        print("Ошибка: Введите корректное числовое значение")
        return None, None

def perform_conversion(choice, value):
    """
    Выполняет конвертацию на основе выбора пользователя

    Args:
        choice (int): Выбор из меню
        value (float): Значение для конвертации

    Returns:
        tuple: (результат, описание)
    """
    conversions = {
        1: (celsius_to_fahrenheit, "°C", "°F"),
        2: (fahrenheit_to_celsius, "°F", "°C"),
        3: (meters_to_feet, "м", "футов"),
        4: (feet_to_meters, "футов", "м"),
        5: (kilograms_to_pounds, "кг", "фунтов"),
        6: (pounds_to_kilograms, "фунтов", "кг"),
        7: (kilometers_to_miles, "км", "миль"),
        8: (miles_to_kilometers, "миль", "км"),
        9: (liters_to_gallons, "литров", "галлонов"),
        10: (gallons_to_liters, "галлонов", "литров")
    }

    if choice in conversions:
        func, from_unit, to_unit = conversions[choice]
        result = func(value)
        description = f"{value} {from_unit} = {result:.2f} {to_unit}"
        return result, description
    else:
        return None, "Неверный выбор"

def main():
    """Основная функция для запуска конвертера"""
    print("Добро пожаловать в конвертер единиц измерения!")

    while True:
        display_conversion_menu()
        choice, value = get_user_input()

        if choice == 0:
            print("Выход из программы. До свидания!")
            break

        if choice is None:
            continue

        result, description = perform_conversion(choice, value)

        if result is not None:
            print(f"Результат: {description}")
        else:
            print("Ошибка: Неверный выбор конвертации")

        input("Нажмите Enter для продолжения...")

if __name__ == "__main__":
    main()

```

Рисунок 4 - Продолжение кода конвертера

ГЛАВА 2. МОДУЛЬНОЕ ТЕСТИРОВАНИЕ

2.1 Введение

Было разработано модульное тестирование для модуля конвертера единиц измерения. Тестирование проводилось с целью проверки корректности работы всех функций конвертации, включая преобразование температур, длин, масс, расстояний и объёмов, а также выявления возможных ошибок в алгоритмах.

2.2 Подход к тестированию

Использован подход white-box тестирования с полным знанием внутренней структуры модуля. Тестирование включало:

- Функциональное тестирование — проверка соответствия функций конвертации заявленным требованиям.
- Тестирование граничных значений — проверка работы с нулевыми, отрицательными и эталонными значениями.
- Тестирование специальных случаев — проверка известных эталонных преобразований ($0^{\circ}\text{C} = 32^{\circ}\text{F}$, $1\text{ м} = 3.28084\text{ футов}$ и т.д.).

2.3 Исходный код модульного тестирования

```
import sys
import os

# Добавляем путь к модулю для импорта
sys.path.append(os.path.dirname(os.path.abspath(__file__)))
from Converter import *

class UnitConverterTester:
    def __init__(self):
        self.total_tests = 0
        self.passed_tests = 0
        self.failed_tests = 0
        self.test_results = []

    def run_test(self, test_function, test_name):
        """Запускает один тест и записывает результат"""
        self.total_tests += 1
        try:
            result = test_function()
            if result:
                self.passed_tests += 1
                self.test_results.append(f"ПРОЙДЕН: {test_name}")
                print(f"ПРОЙДЕН: {test_name}")
                return True
            else:
                self.failed_tests += 1
                self.test_results.append(f"ПРОВАЛЕН: {test_name}")
                print(f"ПРОВАЛЕН: {test_name}")
                return False
        except Exception as e:
            self.failed_tests += 1
            self.test_results.append(f"ОШИБКА: {test_name} - {str(e)}")
            print(f"ОШИБКА: {test_name} - {str(e)}")
            return False

    def assert_approx_equal(self, actual, expected, tolerance=0.01, message=""):
        """Проверяет приблизительное равенство с заданной точностью"""
        if abs(actual - expected) <= tolerance:
            return True
        else:
            print(f" Ожидалось: {expected}, Получено: {actual}, Разница: {abs(actual - expected)}")
            return False

    def test_celsius_to_fahrenheit(self):
        """Тест конвертации Цельсий в Фаренгейт"""
        # Тест 1: 0°C = 32°F
        result1 = celsius_to_fahrenheit(0)
        if not self.assert_approx_equal(result1, 32):
            return False

        # Тест 2: 100°C = 212°F
        result2 = celsius_to_fahrenheit(100)
        if not self.assert_approx_equal(result2, 212):
            return False

        # Тест 3: -40°C = -40°F
        result3 = celsius_to_fahrenheit(-40)
        if not self.assert_approx_equal(result3, -40):
            return False

        # Тест 4: 37°C (температура тела) ≈ 98.6°F
        result4 = celsius_to_fahrenheit(37)
        if not self.assert_approx_equal(result4, 98.6, 0.1):
            return False

        return True
```

Рисунок 5 - Код модульного тестирования

```

def test_fahrenheit_to_celsius(self):
    """Тест конвертации Фаренгейт в Цельсий"""
    # Тест 1: 32°F = 0°C
    result1 = fahrenheit_to_celsius(32)
    if not self.assert_approx_equal(result1, 0):
        return False

    # Тест 2: 212°F = 100°C
    result2 = fahrenheit_to_celsius(212)
    if not self.assert_approx_equal(result2, 100):
        return False

    # Тест 3: -40°F = -40°C
    result3 = fahrenheit_to_celsius(-40)
    if not self.assert_approx_equal(result3, -40):
        return False

    # Тест 4: 98.6°F ≈ 37°C
    result4 = fahrenheit_to_celsius(98.6)
    if not self.assert_approx_equal(result4, 37, 0.1):
        return False

    return True

def test_meters_to_feet(self):
    """Тест конвертации метров в футы"""
    # Тест 1: 1 м = 3.28084 футов
    result1 = meters_to_feet(1)
    if not self.assert_approx_equal(result1, 3.28084):
        return False

    # Тест 2: 10 м = 32.8084 футов
    result2 = meters_to_feet(10)
    if not self.assert_approx_equal(result2, 32.8084):
        return False

    # Тест 3: 0 м = 0 футов
    result3 = meters_to_feet(0)
    if not self.assert_approx_equal(result3, 0):
        return False

    return True

def test_feet_to_meters(self):
    """Тест конвертации футов в метры"""
    # Тест 1: 3.28084 футов = 1 м
    result1 = feet_to_meters(3.28084)
    if not self.assert_approx_equal(result1, 1, 0.0001):
        return False

    # Тест 2: 10 футов ≈ 3.048 м
    result2 = feet_to_meters(10)
    if not self.assert_approx_equal(result2, 3.048, 0.001):
        return False

    # Тест 3: 0 футов = 0 м
    result3 = feet_to_meters(0)
    if not self.assert_approx_equal(result3, 0):
        return False

    return True

def test_kilograms_to_pounds(self):
    """Тест конвертации килограммов в фунты"""
    # Тест 1: 1 кг = 2.20462 фунтов
    result1 = kilograms_to_pounds(1)

```

Рисунок 6 - Продолжение кода модульного тестирования

```

        if not self.assert_approx_equal(result1, 2.20462):
            return False

        # Тест 2: 10 кг = 22.0462 фунтов
        result2 = kilograms_to_pounds(10)
        if not self.assert_approx_equal(result2, 22.0462):
            return False

        # Тест 3: 0 кг = 0 фунтов
        result3 = kilograms_to_pounds(0)
        if not self.assert_approx_equal(result3, 0):
            return False

        return True

def test_pounds_to_kilograms(self):
    """Тест конвертации фунтов в килограммы"""
    # Тест 1: 2.20462 фунтов = 1 кг
    result1 = pounds_to_kilograms(2.20462)
    if not self.assert_approx_equal(result1, 1, 0.0001):
        return False

    # Тест 2: 10 фунтов ≈ 4.53592 кг
    result2 = pounds_to_kilograms(10)
    if not self.assert_approx_equal(result2, 4.53592, 0.001):
        return False

    # Тест 3: 0 фунтов = 0 кг
    result3 = pounds_to_kilograms(0)
    if not self.assert_approx_equal(result3, 0):
        return False

    return True

def test_kilometers_to_miles(self):
    """Тест конвертации километров в мили"""
    # Тест 1: 1 км = 0.621371 миль
    result1 = kilometers_to_miles(1)
    if not self.assert_approx_equal(result1, 0.621371):
        return False

    # Тест 2: 10 км = 6.21371 миль
    result2 = kilometers_to_miles(10)
    if not self.assert_approx_equal(result2, 6.21371):
        return False

    # Тест 3: 0 км = 0 миль
    result3 = kilometers_to_miles(0)
    if not self.assert_approx_equal(result3, 0):
        return False

    return True

def test_miles_to_kilometers(self):
    """Тест конвертации миль в километры"""
    # Тест 1: 0.621371 миль = 1 км
    result1 = miles_to_kilometers(0.621371)
    if not self.assert_approx_equal(result1, 1, 0.0001):
        return False

    # Тест 2: 10 миль = 16.0934 км
    result2 = miles_to_kilometers(10)
    if not self.assert_approx_equal(result2, 16.0934, 0.001):
        return False

```

Рисунок 7 - Продолжение кода модульного тестирования

```

# Тест 3: 0 миль = 0 км
result3 = miles_to_kilometers(0)
if not self.assert_approx_equal(result3, 0):
    return False

return True

def test_liters_to_gallons(self):
    """Тест конвертации литров в галлоны"""
    # Тест 1: 1 литр = 0.264172 галлонов
    result1 = liters_to_gallons(1)
    if not self.assert_approx_equal(result1, 0.264172):
        return False

    # Тест 2: 10 литров = 2.64172 галлонов
    result2 = liters_to_gallons(10)
    if not self.assert_approx_equal(result2, 2.64172):
        return False

    # Тест 3: 0 литров = 0 галлонов
    result3 = liters_to_gallons(0)
    if not self.assert_approx_equal(result3, 0):
        return False

    return True

def test_gallons_to_liters(self):
    """Тест конвертации галлонов в литры"""
    # Тест 1: 0.264172 галлонов = 1 литр
    result1 = gallons_to_liters(0.264172)
    if not self.assert_approx_equal(result1, 1, 0.0001):
        return False

    # Тест 2: 10 галлонов = 37.8541 литров
    result2 = gallons_to_liters(10)
    if not self.assert_approx_equal(result2, 37.8541, 0.001):
        return False

    # Тест 3: 0 галлонов = 0 литров
    result3 = gallons_to_liters(0)
    if not self.assert_approx_equal(result3, 0):
        return False

    return True

def test_perform_conversion(self):
    """Тест функции perform_conversion"""
    # Тест различных типов конвертаций
    test_cases = [
        (1, 0, 32), # Celsius to Fahrenheit
        (2, 32, 0), # Fahrenheit to Celsius
        (3, 1, 3.28084), # Meters to Feet
        (5, 1, 2.20462), # Kilograms to Pounds
        (7, 1, 0.621371), # Kilometers to Miles
        (9, 1, 0.264172), # Liters to Gallons
    ]

    for choice, input_val, expected in test_cases:
        result, description = perform_conversion(choice, input_val)
        if not self.assert_approx_equal(result, expected, 0.001):
            print(f" Ошибка в конвертации типа {choice}")
            return False

```

Рисунок 8 - Продолжение кода модульного тестирования

```

def test_perform_conversion(self):
    """Тест функции perform_conversion"""
    # Тест различных типов конвертаций
    test_cases = [
        (1, 0, 32), # Celsius to Fahrenheit
        (2, 32, 0), # Fahrenheit to Celsius
        (3, 1, 3.28084), # Meters to Feet
        (5, 1, 2.20462), # Kilograms to Pounds
        (7, 1, 0.621371), # Kilometers to Miles
        (9, 1, 0.264172), # Liters to Gallons
    ]

    for choice, input_val, expected in test_cases:
        result, description = perform_conversion(choice, input_val)
        if not self.assert_approx_equal(result, expected, 0.001):
            print(f" Ошибка в конвертации типа {choice}")
            return False

    # Тест неверного выбора
    result, description = perform_conversion(99, 10)
    if result is not None:
        print(" Ожидался None для неверного выбора")
        return False

    return True

def test_edge_cases(self):
    """Тест граничных случаев"""
    # Тест отрицательных значений
    try:
        result1 = celsius_to_fahrenheit(-100)
        result2 = meters_to_feet(-5)
        result3 = kilograms_to_pounds(-10)

        # Проверяем, что функции не падают на отрицательных значениях
        # (не проверяем корректность результата, только отсутствие исключений)
        return True
    except Exception as e:
        print(f" Ошибка при обработке отрицательных значений: {e}")
        return False

def run_all_tests(self):
    """Запускает все тесты и выводит отчет"""
    print("Запуск модульного тестирования конвертера единиц измерения")
    print("=" * 60)

    # Запуск всех тестов
    tests = [
        (self.test_celsius_to_fahrenheit, "Конвертация Цельсий -> Фаренгейт"),
        (self.test_fahrenheit_to_celsius, "Конвертация Фаренгейт -> Цельсий"),
        (self.test_meters_to_feet, "Конвертация метров -> футы"),
        (self.test_feet_to_meters, "Конвертация футов -> метры"),
        (self.test_kilograms_to_pounds, "Конвертация килограммов -> фунты"),
        (self.test_pounds_to_kilograms, "Конвертация фунтов -> килограммы"),
        (self.test_kilometers_to_miles, "Конвертация километров -> мили"),
        (self.test_miles_to_kilometers, "Конвертация миль -> километры"),
        (self.test_liters_to_gallons, "Конвертация литров -> галлоны"),
        (self.test_gallons_to_liters, "Конвертация галлонов -> литры"),
        (self.test_edge_cases, "Граничные случаи"),
    ]

    for test_func, test_name in tests:
        self.run_test(test_func, test_name)

    # Вывод итогового отчета
    print("\n" + "=" * 60)

```

Рисунок 9 - Продолжение кода модульного тестирования

```

print("ИТОГОВЫЙ ОТЧЕТ ТЕСТИРОВАНИЯ")
print("=" * 60)
print(f"Всего тестов: {self.total_tests}")
print(f"Пройдено: {self.passed_tests}")
print(f"Провалено: {self.failed_tests}")

success_rate = (self.passed_tests / self.total_tests) * 100 if self.total_tests > 0 else 0
print(f"Успешность: {success_rate:.1f}%")

if self.failed_tests > 0:
    print("\nДетали проваленных тестов:")
    for result in self.test_results:
        if result.startswith("ПРОВАЛЕН") or result.startswith("ОШИБКА"):
            print(f" - {result}")

return self.failed_tests == 0

def main():
    """Основная функция для запуска тестов"""
    tester = UnitConverterTester()
    success = tester.run_all_tests()

    if success:
        print("\nСтатус: Все тесты пройдены успешно!")
        sys.exit(0)
    else:
        print("\nСтатус: Обнаружены ошибки в модуле!")
        sys.exit(1)

if __name__ == "__main__":
    main()

```

Рисунок 10 - Продолжение кода модульного тестирования

2.4 Используемые техники

- Модульное тестирование отдельных функций конвертации.
- Интеграционное тестирование взаимодействия функций через `perform_conversion`.
- Тестирование с использованием утверждений (assertions) с допустимой погрешностью.
- Параметризованное тестирование с различными наборами данных.

2.5 Результаты тестирования

Была обнаружена преднамеренная ошибка в функции `kilograms_to_pounds` — неправильный коэффициент конвертации. В процессе тестирования выяснилось, что код функции фактически содержал неверный коэффициент (2.25

вместо 2.20462). Все тестовые случаи помимо связанного с kilograms_to_pounds были успешно пройдены, включая проверку:

- Эталонных значений (0°C, 100°C, -40°C и др.).
- Обратных преобразований (футы в метры, фунты в килограммы и т.д.).
- Граничных случаев (отрицательные значения, нулевые входные данные).
- Результаты тестирования показали 90.9% успешность выполнения тестов.

```
Запуск модульного тестирования конвертера единиц измерения
=====
ПРОЙДЕН: Конвертация Цельсий -> Фаренгейт
ПРОЙДЕН: Конвертация Фаренгейт -> Цельсий
ПРОЙДЕН: Конвертация метров -> футы
ПРОЙДЕН: Конвертация футов -> метры
    Ожидалось: 2.20462, Получено: 2.25, Разница: 0.04538000000000002
ПРОВАЛЕН: Конвертация килограммов -> фунты
ПРОЙДЕН: Конвертация фунтов -> килограммы
ПРОЙДЕН: Конвертация километров -> мили
ПРОЙДЕН: Конвертация миль -> километры
ПРОЙДЕН: Конвертация литров -> галлоны
ПРОЙДЕН: Конвертация галлонов -> литры
ПРОЙДЕН: Граничные случаи

=====
ИТОГОВЫЙ ОТЧЕТ ТЕСТИРОВАНИЯ
=====
Всего тестов: 11
Пройдено: 10
Провалено: 1
Успешность: 90.9%

Детали проваленных тестов:
- ПРОВАЛЕН: Конвертация килограммов -> фунты

Статус: Обнаружены ошибки в модуле!
```

Рисунок 11 - Изначальные результаты тестирования

После исправления кода, тестирование прошло успешно, все тестовые случаи были пройдены.

```
Запуск модульного тестирования конвертера единиц измерения
=====
ПРОЙДЕН: Конвертация Цельсий -> Фаренгейт
ПРОЙДЕН: Конвертация Фаренгейт -> Цельсий
ПРОЙДЕН: Конвертация метров -> футы
ПРОЙДЕН: Конвертация футов -> метры
ПРОЙДЕН: Конвертация килограммов -> фунты
ПРОЙДЕН: Конвертация фунтов -> килограммы
ПРОЙДЕН: Конвертация километров -> мили
ПРОЙДЕН: Конвертация миль -> километры
ПРОЙДЕН: Конвертация литров -> галлоны
ПРОЙДЕН: Конвертация галлонов -> литры
ПРОЙДЕН: Граничные случаи

=====
ИТОГОВЫЙ ОТЧЕТ ТЕСТИРОВАНИЯ
=====
Всего тестов: 11
Пройдено: 11
Провалено: 0
Успешность: 100.0%

Статус: Все тесты пройдены успешно!
```

Рисунок 12 - Результаты тестирования

ГЛАВА 3. МУТАЦИОННОЕ ТЕСТИРОВАНИЕ

3.1 Введение

Было проведено мутационное тестирование модуля конвертера единиц измерения с целью оценки эффективности существующих модульных тестов и выявления потенциальных слабых мест в тестовом покрытии.

3.2 Подход к тестированию

Использована стратегия мутационного тестирования с применением различных типов мутационных операторов, включая базовые и расширенные мутации.

3.3 Исходный код мутационного тестирования

```
import sys
import os
import ast
import copy
import inspect
import random
from typing import List, Dict, Any

# Добавляем путь к модулю для импорта
sys.path.append(os.path.dirname(os.path.abspath(__file__)))
from Converter import *

class MutationTester:
    def __init__(self):
        self.total_mutants = 0
        self.killed_mutants = 0
        self.survived_mutants = 0
        self.mutation_results = []
        self.original_functions = {}

    def save_original_functions(self):
        """Сохраняет оригинальные версии функций"""
        functions_to_test = [
            'celsius_to_fahrenheit', 'fahrenheit_to_celsius',
            'meters_to_feet', 'feet_to_meters',
            'kilograms_to_pounds', 'pounds_to_kilograms',
            'kilometers_to_miles', 'miles_to_kilometers',
            'liters_to_gallons', 'gallons_to_liters'
        ]

        for func_name in functions_to_test:
            self.original_functions[func_name] = globals()[func_name]

    def restore_original_functions(self):
        """Восстанавливает оригинальные функции"""
        for func_name, func in self.original_functions.items():
            globals()[func_name] = func

    def create_test_cases(self) -> List[Dict[str, Any]]:
        """Создает тестовые случаи для проверки мутантов"""
        return [
            # celsius_to_fahrenheit
            {'function': 'celsius_to_fahrenheit', 'input': 0, 'expected': 32},
            {'function': 'celsius_to_fahrenheit', 'input': 100, 'expected': 212},
            {'function': 'celsius_to_fahrenheit', 'input': -40, 'expected': -40},

            # fahrenheit_to_celsius
            {'function': 'fahrenheit_to_celsius', 'input': 32, 'expected': 0},
            {'function': 'fahrenheit_to_celsius', 'input': 212, 'expected': 100},
            {'function': 'fahrenheit_to_celsius', 'input': -40, 'expected': -40},

            # meters_to_feet
            {'function': 'meters_to_feet', 'input': 1, 'expected': 3.28084},
            {'function': 'meters_to_feet', 'input': 10, 'expected': 32.8084},

            # feet_to_meters
            {'function': 'feet_to_meters', 'input': 3.28084, 'expected': 1},
            {'function': 'feet_to_meters', 'input': 10, 'expected': 3.048},

            # kilograms_to_pounds
            {'function': 'kilograms_to_pounds', 'input': 1, 'expected': 2.20462},
            {'function': 'kilograms_to_pounds', 'input': 10, 'expected': 22.0462},

            # pounds_to_kilograms
            {'function': 'pounds_to_kilograms', 'input': 2.20462, 'expected': 1},
            {'function': 'pounds_to_kilograms', 'input': 10, 'expected': 4.53592},
```

Рисунок 13 - Исходный код мутационного тестирования

```

    # kilometers_to_miles
    {'function': 'kilometers_to_miles', 'input': 1, 'expected': 0.621371},
    {'function': 'kilometers_to_miles', 'input': 10, 'expected': 6.21371},

    # miles_to_kilometers
    {'function': 'miles_to_kilometers', 'input': 0.621371, 'expected': 1},
    {'function': 'miles_to_kilometers', 'input': 10, 'expected': 16.0934},

    # liters_to_gallons
    {'function': 'liters_to_gallons', 'input': 1, 'expected': 0.264172},
    {'function': 'liters_to_gallons', 'input': 10, 'expected': 2.64172},

    # gallons_to_liters
    {'function': 'gallons_to_liters', 'input': 0.264172, 'expected': 1},
    {'function': 'gallons_to_liters', 'input': 10, 'expected': 37.8541},
]

def run_test_suite(self, test_cases: List[Dict[str, Any]]) -> bool:
    """Запускает тестовый набор для текущей версии функций"""
    try:
        for test_case in test_cases:
            func_name = test_case['function']
            input_val = test_case['input']
            expected = test_case['expected']

            func = globals()[func_name]
            result = func(input_val)

            # Проверка с допуском 0.001
            if abs(result - expected) > 0.001:
                return False
        return True
    except Exception:
        return False

def mutate_arithmetic_operator(self, func):
    """Мутация арифметических операторов"""
    original_source = inspect.getsource(func)

    mutations = [
        ('+', '-'), ('-', '+'), ('*', '/'), ('/', '*'),
        ('+', '*'), ('-', '/'), ('*', '+'), ('/', '-')
    ]

    mutated_funcs = []
    for old_op, new_op in mutations:
        if old_op in original_source:
            mutated_source = original_source.replace(old_op, new_op)
            try:
                # Создаем мутированную функцию
                exec(mutated_source, globals())
                mutated_func_name = func.__name__
                mutated_funcs.append(globals()[mutated_func_name])
            except:
                continue

    return mutated_funcs

def mutate_numeric_values(self, func):
    """Мутация числовых значений"""
    original_source = inspect.getsource(func)

    # Ищем числовые значения в коде
    tree = ast.parse(original_source)

```

Рисунок 15 - Продолжение исходного кода мутационного тестирования

```

def mutate_numeric_values(self, func):
    numbers = []

    for node in ast.walk(tree):
        if isinstance(node, ast.Constant) and isinstance(node.value, (int, float)):
            numbers.append(node.value)
        elif isinstance(node, ast.Num): # Для старых версий Python
            numbers.append(node.n)

    mutated_funcs = []
    for number in set(numbers):
        if number != 0: # Не мутуируем нули
            # Мутации: инкремент, декремент, инверсия знака
            mutations = [
                number + 1, number - 1, number * 2, number / 2,
                -number, number + 0.1, number - 0.1
            ]

            for mutated_num in mutations:
                old_str = str(number)
                new_str = str(mutated_num)

                if old_str in original_source:
                    mutated_source = original_source.replace(old_str, new_str)
                    try:
                        exec(mutated_source, globals())
                        mutated_func_name = func.__name__
                        mutated_funcs.append(globals()[mutated_func_name])
                    except:
                        continue

    return mutated_funcs

def mutate_comparison_operators(self, func):
    """Мутация операторов сравнения (если есть)"""
    original_source = inspect.getsource(func)

    mutations = [
        ('<', '>'), ('>', '<'), ('<=', '>='), ('>=', '<='),
        ('==', '!='), ('!=', '==')
    ]

    mutated_funcs = []
    for old_op, new_op in mutations:
        if old_op in original_source:
            mutated_source = original_source.replace(old_op, new_op)
            try:
                exec(mutated_source, globals())
                mutated_func_name = func.__name__
                mutated_funcs.append(globals()[mutated_func_name])
            except:
                continue

    return mutated_funcs

def mutate_function(self, func_name: str):
    """Создает мутации для указанной функции"""
    func = globals()[func_name]
    mutations = []

    # Сохраняем оригинальную функцию
    original_func = globals()[func_name]

    # Применяем различные типы мутаций
    mutations.extend(self.mutate_arithmetic_operator(func))
    mutations.extend(self.mutate_numeric_values(func))

```

Рисунок 16 - Продолжение исходного кода мутационного тестирования

```

mutations.extend(self.mutate_comparison_operators(func))

# Восстанавливаем оригинальную функцию
globals()[func_name] = original_func

return mutations

def test_mutant(self, mutant_func, func_name: str, test_cases: List[Dict[str, Any]]) -> bool:
    """Тестирует одного мутанта"""
    # Сохраняем текущую функцию
    current_func = globals()[func_name]

    try:
        # Заменяем функцию мутантом
        globals()[func_name] = mutant_func

        # Запускаем тесты
        test_passed = self.run_test_suite(test_cases)

        return test_passed
    finally:
        # Восстанавливаем оригинальную функцию
        globals()[func_name] = current_func

def run_mutation_testing(self):
    """Запускает мутационное тестирование"""
    print("Запуск мутационного тестирования конвертера единиц измерения")
    print("=" * 70)

    self.save_original_functions()
    test_cases = self.create_test_cases()

    functions_to_mutate = list(self.original_functions.keys())

    for func_name in functions_to_mutate:
        print(f"\nМутационное тестирование функции: {func_name}")
        print("-" * 50)

        mutants = self.mutate_function(func_name)
        print(f"Создано мутантов: {len(mutants)}")

        for i, mutant in enumerate(mutants):
            self.total_mutants += 1

            try:
                survived = self.test_mutant(mutant, func_name, test_cases)

                if survived:
                    self.survived_mutants += 1
                    status = "ВЫЖИЛ"
                else:
                    self.killed_mutants += 1
                    status = "УБИТ"

                result_msg = f" Мутант {i + 1}: {status}"
                self.mutation_results.append(f"{func_name} - {result_msg}")
                print(result_msg)

            except Exception as e:
                self.total_mutants -= 1 # Не считаем мутантов, вызвавших ошибку
                print(f" Мутант {i + 1}: ОШИБКА - {str(e)}")

    # Восстанавливаем оригинальные функции
    self.restore_original_functions()

```

Рисунок 17 - Продолжение исходного кода мутационного тестирования

```

def run_mutation_testing(self):
    self.generate_report()

def generate_report(self):
    """Генерирует отчет о мутационном тестировании"""
    print("\n" + "=" * 70)
    print("ОТЧЕТ МУТАЦИОННОГО ТЕСТИРОВАНИЯ")
    print("=" * 70)

    print(f"Всего создано мутантов: {self.total_mutants}")
    print(f"Убито мутантов: {self.killed_mutants}")
    print(f"Выжило мутантов: {self.survived_mutants}")

    if self.total_mutants > 0:
        mutation_score = (self.killed_mutants / self.total_mutants) * 100
        print(f"Мутационный счет: {mutation_score:.1f}%")

    print("\nДетали результатов:")
    for result in self.mutation_results[:20]: # Показываем первые 20 результатов
        print(f" {result}")

    if len(self.mutation_results) > 20:
        print(f" ... и еще {len(self.mutation_results) - 20} результатов")

    # Анализ качества тестов
    if self.total_mutants > 0:
        if mutation_score >= 80:
            print("\nСтатус: Отличное покрытие мутациями!")
        elif mutation_score >= 60:
            print("\nСтатус: Хорошее покрытие мутациями")
        elif mutation_score >= 40:
            print("\nСтатус: Удовлетворительное покрытие мутациями")
        else:
            print("\nСтатус: Низкое покрытие мутациями - нужны улучшения тестов")

class AdvancedMutationTester(MutationTester):
    """Расширенный мутационный тестер с дополнительными мутациями"""

    def mutate_logical_operators(self, func):
        """Мутация логических операторов"""
        original_source = inspect.getsource(func)

        mutations = [
            ('and', 'or'), ('or', 'and'),
            ('not', ''), ('', 'not ') # Упрощенная версия
        ]

        mutated_funcs = []
        for old_op, new_op in mutations:
            if old_op in original_source and old_op.strip():
                mutated_source = original_source.replace(old_op, new_op)
                try:
                    exec(mutated_source, globals())
                    mutated_func_name = func.__name__
                    mutated_funcs.append(globals()[mutated_func_name])
                except:
                    continue

        return mutated_funcs

    def mutate_assignment_operators(self, func):
        """Мутация операторов присваивания"""
        original_source = inspect.getsource(func)

        mutations = [

```

Рисунок 18 - Продолжение исходного кода мутационного тестирования

```

        ('=', '=='), ('==', '='),
        ('+=', '-='), ('-=', '+='),
        ('*=', '/='), ('/=', '*=')
    ]

    mutated_funcs = []
    for old_op, new_op in mutations:
        if old_op in original_source:
            mutated_source = original_source.replace(old_op, new_op)
            try:
                exec(mutated_source, globals())
                mutated_func_name = func.__name__
                mutated_funcs.append(globals()[mutated_func_name])
            except:
                continue

    return mutated_funcs

def mutate_function_calls(self, func):
    """Мутация вызовов функций"""
    original_source = inspect.getsource(func)

    # Простая мутация - замена имен функций на похожие
    function_replacements = [
        ('abs', 'round'), ('round', 'abs'),
        ('int', 'float'), ('float', 'int')
    ]

    mutated_funcs = []
    for old_func, new_func in function_replacements:
        if old_func in original_source:
            mutated_source = original_source.replace(old_func, new_func)
            try:
                exec(mutated_source, globals())
                mutated_func_name = func.__name__
                mutated_funcs.append(globals()[mutated_func_name])
            except:
                continue

    return mutated_funcs

def mutate_function(self, func_name: str):
    """Расширенная версия создания мутаций"""
    func = globals()[func_name]
    mutations = []

    original_func = globals()[func_name]

    # Базовые мутации
    mutations.extend(self.mutate_arithmetic_operator(func))
    mutations.extend(self.mutate_numeric_values(func))
    mutations.extend(self.mutate_comparison_operators(func))

    # Расширенные мутации
    mutations.extend(self.mutate_logical_operators(func))
    mutations.extend(self.mutate_assignment_operators(func))
    mutations.extend(self.mutate_function_calls(func))

    globals()[func_name] = original_func

    return mutations

```

Рисунок 19 - Продолжение исходного кода мутационного тестирования

```

def main():
    """Основная функция для запуска мутационного тестирования"""
    print("МУТАЦИОННОЕ ТЕСТИРОВАНИЕ КОНВЕРТЕРА ЕДИНИЦ ИЗМЕРЕНИЯ")
    print("=" * 70)

    # Проверяем, что оригинальные тесты проходят
    print("Проверка оригинальных тестов...")
    tester = MutationTester()
    test_cases = tester.create_test_cases()

    if tester.run_test_suite(test_cases):
        print("✓ Оригинальные тесты проходят")
    else:
        print("X Оригинальные тесты не проходят! Прерывание.")
        return

    print("\nЗапуск базового мутационного тестирования...")
    basic_tester = MutationTester()
    basic_tester.run_mutation_testing()

    print("\n" + "=" * 70)
    print("ЗАПУСК РАСШИРЕННОГО МУТАЦИОННОГО ТЕСТИРОВАНИЯ")
    print("=" * 70)

    advanced_tester = AdvancedMutationTester()
    advanced_tester.run_mutation_testing()

if __name__ == "__main__":
    main()

```

Рисунок 20 - Продолжение исходного кода мутационного тестирования

3.4 Типы мутационных операторов

- Базовые операторы:
 - Изменение арифметических операторов (+ на -, * на / и др.).
 - Модификация числовых значений (изменение коэффициентов конвертации).

- Изменение операторов сравнения (если присутствуют).
- Расширенные операторы:
 - Мутации логических операторов (and/or).
 - Мутации операторов присваивания (=, += и др.).
 - Мутации вызовов функций.

3.5 Результаты тестирования

Было запущено мутационное тестирование. В итоге вывело данную информацию (Рисунок 21)

```
=====
ОТЧЕТ МУТАЦИОННОГО ТЕСТИРОВАНИЯ
=====
Всего создано мутантов: 136
Убито мутантов: 126
Выжило мутантов: 10
Мутационный счет: 92.6%

Детали результатов:
celsius_to_fahrenheit - Мутант 1: УБИТ
celsius_to_fahrenheit - Мутант 2: УБИТ
celsius_to_fahrenheit - Мутант 3: УБИТ
celsius_to_fahrenheit - Мутант 4: УБИТ
celsius_to_fahrenheit - Мутант 5: УБИТ
celsius_to_fahrenheit - Мутант 6: УБИТ
celsius_to_fahrenheit - Мутант 7: УБИТ
celsius_to_fahrenheit - Мутант 8: УБИТ
celsius_to_fahrenheit - Мутант 9: УБИТ
celsius_to_fahrenheit - Мутант 10: УБИТ
celsius_to_fahrenheit - Мутант 11: УБИТ
celsius_to_fahrenheit - Мутант 12: УБИТ
celsius_to_fahrenheit - Мутант 13: УБИТ
celsius_to_fahrenheit - Мутант 14: УБИТ
celsius_to_fahrenheit - Мутант 15: УБИТ
celsius_to_fahrenheit - Мутант 16: УБИТ
celsius_to_fahrenheit - Мутант 17: УБИТ
celsius_to_fahrenheit - Мутант 18: УБИТ
celsius_to_fahrenheit - Мутант 19: УБИТ
celsius_to_fahrenheit - Мутант 20: УБИТ
... и еще 116 результатов

Статус: Отличное покрытие мутациями!
```

Рисунок 21 - Результаты мутационного тестирования

Мутационное тестирование выявило, что тестовое покрытие является достаточно качественным. Большинство мутантов были успешно обнаружены, особенно в области изменений арифметических операторов и числовых значений. После доработки исходного кода мутационный счет составил высокий процент, что свидетельствует о хорошей эффективности тестов.

3.6 Анализ выживших мутантов

- Мутации в числовых константах: некоторые изменения коэффициентов не были обнаружены из-за ограниченного набора тестовых значений.
- Мутации арифметических операторов: в редких случаях замена оператора не влияла на конечный результат в пределах допустимой погрешности.

ГЛАВА 4. ВЫВОД

4.1 Модульное тестирование

Показатели эффективности:

- Покрытие функциональности: 100% — все функции модуля протестированы.
- Успешность выполнения: 100% — все тесты прошли успешно.
- Тестовые сценарии: комплексное покрытие различных типов конвертаций.

Сильные стороны:

- Полное покрытие всех функций конвертации.
- Тестирование граничных случаев и эталонных значений.
- Проверка согласованности между прямыми и обратными преобразованиями.

4.2 Мутационное тестирование

Показатели эффективности:

- Высокий процент обнаружения мутаций после доработки тестов.
- Систематическое покрытие всех критических функций.

Преимущества примененного подхода:

- Фокус на корректности формул конвертации.
- Автоматизированный процесс тестирования.
- Интеграция с существующим набором модульных тестов.

4.3 Анализ недостатков и их устранение

Выявленные проблемы:

- Неполное покрытие мутаций числовых констант
- Недостаточное тестирование арифметических операторов — добавлены тесты, проверяющие корректность формул.

4.4 Итоговые выводы

Общий статус качества: ОТЛИЧНО

Модуль конвертера единиц измерения прошел комплексное тестирование и демонстрирует:

- Функциональную корректность.
- Высокую тестируемость кода.
- Устойчивость к регрессионным изменениям.
- Соответствие базовым требованиям к конвертации единиц.

ЗАКЛЮЧЕНИЕ

В ходе выполнения работы был проведен комплексный анализ модуля конвертера единиц измерения и соответствующего тестового обеспечения. Мутационное тестирование подтвердило важность тщательного тестирования функций конвертации, особенно в контексте точности формул и коэффициентов.

Обнаруженная и исправленная ошибка в коде подчеркивает важность поддержания документации в актуальном состоянии. Дальнейшие улучшения могут включать добавление проверки на большем количестве тестовых данных и реализацию более сложных сценариев тестирования, таких как проверка комбинированных преобразований и обработки исключительных ситуаций.